

THEORY NOTES

C++ Memory Management

From raw pointers and the new/delete dance to
RAII and modern smart pointers

Stack vs Heap

Raw Pointers

RAII

unique_ptr

shared_ptr

Compiled from video transcript · youtube.com/watch?v=QuR24dR9L2g

01 Why Manual Memory Management Exists

Languages like Python or Java have a **garbage collector** running quietly in the background, automatically finding and freeing memory you no longer need. C++ makes a different bet: it hands **direct control over every single byte** to the programmer. No background process, no automatic cleanup, no safety net — unless you build one yourself.

This sounds risky, and it is, but it's also exactly why C++ dominates domains where every millisecond and every megabyte counts: **video games, operating systems, game engines, and embedded systems**. A garbage collector can pause your program at an unpredictable moment to clean up — unacceptable when you're rendering 60+ frames a second or running a kernel. C++ trades that convenience for raw, predictable performance.

★ NOTE

The rest of these notes build up, piece by piece, from "raw and dangerous" (manual pointers) to "safe and modern" (smart pointers) — the same journey the C++ language itself took over the last ~15 years.

02 Where Data Lives: The Stack and The Heap

Every variable in a C++ program is stored in one of two places, and understanding the difference is the foundation for everything else in this document.

| The Stack

The Stack is small, fast, and tidy. Every time you enter a function or a block of code `{ ... }`, variables declared there are pushed onto the stack. The moment that block ends, those variables are **automatically popped off and destroyed** — no effort required from you.

| The Heap

The Heap is a much larger pool of memory that doesn't care about function scope. Anything you put there sticks around until *you* explicitly remove it — even after the function that created it has finished running. This flexibility is powerful, but it comes with a responsibility: the heap has no automatic cleanup.

🧠 ANALOGY

Think of the **Stack** as a small desk in a meeting room: whatever you bring in gets cleared away automatically the moment you leave the room. The **Heap** is a giant warehouse across town: you can store as much as you like for as long as you like, but nobody is going to clean it up for you. If you rent a shelf there, it's your job to empty it when you're done.

Property	Stack	Heap
Size	Small (often a few MB)	Large (limited mainly by system RAM)
Speed	Very fast allocation	Slower allocation
Lifetime	Tied to scope (block/function)	Lives until explicitly freed
Cleanup	Automatic	Manual (<code>delete</code>) or smart pointer
Risk	Stack overflow if too much is pushed	Leaks, dangling pointers, double-free
Typical use	Local variables, small fixed-size data	Large or dynamically-sized data, data that must outlive a function

03 Raw Pointers: The Treasure Map to the Heap

A **pointer** doesn't hold a value directly — it holds the **memory address** of where a value lives. If the Heap is a warehouse, a pointer is the slip of paper with the shelf number written on it.

🗝️ ANALOGY

A raw pointer is a **treasure map**. The treasure (your data) is buried out in the Heap. The map (the pointer) just tells you where to dig. Lose the map, and the treasure is still out there — just unreachable and wasting space forever.

Working with heap memory always follows the same mandatory three-step routine:

The Three-Step Dance

- **1. Request** — ask the operating system for memory using `new`
- **2. Use** — read from / write to that memory through the pointer
- **3. Release** — manually give the memory back using `delete`

memory_dance.cpp

```
// Step 1: Request memory on the heap
int* score = new int(42);

// Step 2: Use it (dereference with *)
std::cout << "Score: " << *score << std::endl;
*score = 100; // modify the value at that address

// Step 3: Release it manually
delete score;
```

Here, `score` itself is a small variable living on the **Stack** (it will be cleaned up automatically), but it points to an `int` living on the **Heap** (which only disappears when you call `delete`).

04 The Three Deadly Sins of Manual Memory

Forgetting any step in the three-step dance — or doing an extra step you shouldn't — leads to one of three classic bugs. These are the bugs that give manual memory management its scary reputation.

1. Memory Leak

⚠ DANGER — MEMORY LEAK

You call `new` but never call the matching `delete`. The memory stays "checked out" forever, even though nothing in your program can reach it anymore. Do this repeatedly (e.g. inside a loop) and your program's memory usage climbs until the system runs out.

leak.cpp

```
void loadLevel() {
    int* data = new int(99);
    // ... function ends, "data" (the map) is gone
    // but the memory it pointed to is never freed -- LEAKED
}
```

2. Dangling Pointer

⚠ WARNING — DANGLING POINTER

You delete the memory, but keep using the pointer as if it were still valid. The address it holds no longer belongs to you — it might now contain garbage, or even be reassigned to something else entirely. Reading or writing through it causes **undefined behavior**.

dangling.cpp

```
int* ptr = new int(7);
delete ptr;           // memory freed
std::cout << *ptr;    // DANGER: reading freed memory
```

3. Double Delete

⚠ DANGER — DOUBLE DELETE

You call `delete` on the same pointer twice. The second call tries to free memory that's already been returned — this typically causes an **immediate crash** or corrupts the program's memory bookkeeping.

double_delete.cpp

```
int* ptr = new int(5);
delete ptr;
delete ptr; // CRASH: freeing already-freed memory
```

Bug	What happens	Typical symptom
Memory Leak	<code>new</code> without matching <code>delete</code>	Memory usage grows over time
Dangling Pointer	Using a pointer after its memory was deleted	Garbage values, random crashes
Double Delete	<code>delete</code> called twice on the same pointer	Immediate crash / heap corruption

05 Constructors, Destructors & RAII

Before we can fix the problems above, we need two building blocks that C++ runs **automatically**:

- **Constructor** — a special function that runs the instant an object is created. Used for setup.
- **Destructor** — a special function that runs the instant an object goes out of scope (leaves the Stack). Used for cleanup.

Here's the key insight that fixes manual memory management: if an object's **destructor** contains a `delete` call, then heap cleanup happens **automatically** the moment the object's scope ends — you get heap-like flexibility with stack-like automatic cleanup. This pairing of "acquire a resource in the constructor, release it in the destructor" is a foundational C++ paradigm called **RAII** (Resource Acquisition Is Initialization).

raii_example.cpp

```
class IntBox {
public:
    int* data;

    // Constructor: runs automatically on creation
    IntBox(int value) {
        data = new int(value);
    }

    // Destructor: runs automatically when object goes out of scope
    ~IntBox() {
        delete data;
        std::cout << "Cleaned up automatically!";
    }
};

void useBox() {
    IntBox box(10);    // constructor runs -> heap memory allocated
    // ... use box.data ...
} // box leaves scope here -> destructor runs -> delete happens automatically
```

TIP

RAII isn't just for memory. The exact same pattern is used in real C++ codebases for file handles (close the file in the destructor), mutex locks (unlock in the destructor), and network connections (disconnect in the destructor). Anywhere a resource needs guaranteed cleanup, RAII is the go-to tool.

06 Smart Pointers: RAII, Pre-Packaged

Writing your own RAII wrapper class every time you need heap memory would get repetitive fast. Modern C++ (C++11 and later) solves this by providing ready-made RAII wrapper templates called **smart pointers**. They behave like raw pointers (you can dereference them, access members, etc.) but their destructor automatically calls `delete` for you — you never write `delete` yourself again.

There are two main smart pointers to know, and they differ in one key question: **how many owners is the data allowed to have at once?**

`unique_ptr` — One Owner, No Sharing

`std::unique_ptr` enforces that exactly **one** pointer owns the memory at any given time. It cannot be copied (only "moved"), which guarantees there's never any ambiguity about who is responsible for cleanup.

unique_ptr.cpp

```
#include <memory>

std::unique_ptr<int> ptr = std::make_unique<int>(42);
std::cout << *ptr; // use it just like a raw pointer

// No delete needed -- the destructor handles it automatically
// when "ptr" goes out of scope.
```

`shared_ptr` — Many Owners, Reference Counted

`std::shared_ptr` allows **multiple** pointers to co-own the same heap memory. Internally it keeps a reference count of how many `shared_ptr`s currently point at the data. Each time one is copied, the count goes up; each time one is destroyed, the count goes down. The actual memory is only freed when that count finally hits **zero** — i.e., the last owner releases it.

shared_ptr.cpp

```
#include <memory>

std::shared_ptr<int> a = std::make_shared<int>(42);
std::shared_ptr<int> b = a; // b co-owns the same memory, count = 2

std::cout << a.use_count(); // prints 2

// memory is only deleted once BOTH a and b go out of scope
```

Aspect	unique_ptr	shared_ptr
Ownership	Exactly one owner	Multiple co-owners (reference counted)
Copyable?	No (move-only)	Yes, freely copyable
Overhead	Essentially zero — as fast as a raw pointer	Small overhead for the reference counter
Deleted when	The single owner goes out of scope	The reference count reaches zero
Create with	<code>std::make_unique<T>(...)</code>	<code>std::make_shared<T>(...)</code>
Best for	Default choice — single clear owner	Data genuinely shared across multiple parts of a program

✦ **NOTE — WORTH KNOWING (BEYOND THIS VIDEO)**

A third smart pointer, `std::weak_ptr`, often travels alongside `shared_ptr`. It can observe shared data **without** increasing the reference count, which helps avoid "reference cycles" where two `shared_ptr`s point at each other and neither count ever reaches zero.

07 The Modern C++ Philosophy

Putting it all together, modern C++ has settled on a clear philosophy: **safety by default**. Rather than reaching for raw `new` / `delete` and risking leaks, dangling pointers, or double deletes, idiomatic modern code reaches for smart pointers first. They cost little to nothing in performance (especially `unique_ptr`) while removing an entire category of bugs.

This frees developers to spend their energy on what actually matters in performance-critical C++ work — algorithms, data layout, and speed — instead of hunting down crashes caused by a missing `delete`.

The Big Picture, In One Line

- **Stack** = automatic, fast, small | **Heap** = manual, flexible, large
- **Raw pointers** require the `new` → use → `delete` dance, and any misstep causes a leak, dangling pointer, or crash
- **RAII** ties cleanup to an object's lifetime via constructors/destructors
- **Smart pointers** are pre-built RAII wrappers: `unique_ptr` for single ownership, `shared_ptr` for shared ownership
- Modern C++ defaults to smart pointers; raw `new` / `delete` is the exception, not the rule

08 Video Timestamp Reference

Jump back to any part of the original video for a refresher:

00:00 Why C++ uses manual memory management — the power/performance trade-off

01:13 Stack vs Heap: where data lives

01:49 Raw pointers and the new / use / delete dance

02:55 Memory leaks, dangling pointers, and double deletes

03:33 Constructors, destructors, and RAI

04:41 Introduction to smart pointers

05:43 `unique_ptr` vs `shared_ptr`

06:05 The modern C++ philosophy: safety by default

Theory notes generated from video transcript · for personal study use